MATH 345 Skeleton Notes
Unit 1: Vectors, matrices, and linear maps
Section 1.5: Applications and computation recap

About `@`

The same symbol `@` can mean different linear-algebra operations. Its meaning depends on the shapes of the objects being multiplied.
A short computation involving `@` should be read in three steps.

1. Identify the objects.
2. Identify the operation.
3. Interpret the output.

- **Code fragment:** `u @ v`
  First question to ask: Are `u` and `v` vectors?
  Typical interpretation: dot product; alignment or score
- **Code fragment:** `A @ x`
  First question to ask: What are the rows and columns of `A`?
  Typical interpretation: matrix-vector product
- **Code fragment:** `X @ q`
  First question to ask: Are rows of `X` being scored by `q`?
  Typical interpretation: several dot products at once
- **Code fragment:** `K @ q`
  First question to ask: Which earlier activity computed these dot products by hand?
  Typical interpretation: token-query scores

- **Code fragment:** `alpha @ V`
  First question to ask: Which vectors are being averaged?
  Typical interpretation: weighted average of rows of `V`

- **Code fragment:** `A @ X`
  First question to ask: Are columns of `X` input vectors?
  Typical interpretation: one matrix applied to many inputs

- **Code fragment:** `Q @ K.T`
  First question to ask: What are the query-key scores?
  Typical interpretation: all query-key dot products

- **Code fragment:** `W @ x + b`
  First question to ask: What is $f(\mathbf{0})$?
  Typical interpretation: affine map; linear only when $\mathbf{b} = \mathbf{0}$

Review activities

Activity: Reading a ranking

Continuing the cosine-similarity ranking from The same ranking in code, the code is

```
scores = np.array([1.0, 0.5, 0.0])
doc_names = np.array(["D1", "D2", "D3"])
ranking = doc_names[np.argsort(scores)[::-1]]
ranking
```

1. What is the value of `ranking`?
2. What is being ranked?
3. Is this ranking based on distance or direction?

Activity: Reading an attention computation

In The same attention calculation in code, the code was

```
s = K @ q
alpha = s / s.sum()
out = alpha @ V
```

1. What does `s` contain?
2. Why do the entries of `alpha` add to 1?
3. What does `out` represent?

Activity: Reading a geometric computation

In Applying one matrix to many points, the code was

```
X = np.array([
    [0, 1, 1, 0, 0],
    [0, 0, 1, 1, 0],
], dtype=float)
```

Activity: Reading a geometric computation (continued)

```
A = np.array([
    [1, 1],
    [0, 1],
], dtype=float)

Y = A @ X
Y
```

1. What are the columns of `X`?
2. What are the columns of `Y`?
3. Which transformation from the gallery is this?

Activity: Reading composition order

```
S = np.array([
    [2., 0.],
    [0., 1.],
])

R = np.array([
    [0., -1.],
    [1.,  0.],
])
```

Activity: Reading composition order (continued)

```
R @ S, S @ R
```

1. Which product applies `S` first and then `R`?
2. Are the two products equal?
3. What does the difference mean geometrically?

Activity: Reading an affine rule

```
W = np.array([
    [1., 1.],
    [0., 2.],
])

b = np.array([3., -1.])

def f(x):
    return W @ x + b
```

Is `f` linear? Justify your answer using the zero vector.

Linked notebook

Run Lab U1: Vectors, similarity, attention, and matrix actions. The core path practices vector applications, document similarity, token attention, matrix-vector products, geometric matrix actions, applying one matrix to many points, composition, affine maps, and short code outputs. The final part contains review questions and a short forward look to square-grid visualizations.

For a quick reference on arrays, matrix products, norms, rankings, and slicing, see the programming appendix sections NumPy Arrays, Vectors, Matrices, and Shapes, Elementwise Arithmetic Versus Linear Algebra, Dot Products, Norms, Distances, and Cosine Similarity, Sorting and Rankings, and Indexing and Slicing.